CSCE 45503 Information Retrieval
Homework 3
By: Timothy Pham and Haley Trejo

Objectives

The objective of this assignment was to build an inverted index for a collection of HTML documents using Flex and C++. The program processes documents, counts term frequencies, and stores the results in dictionary, posting, and mapping files. It also reduces the size of the index by filtering long tokens and removing terms that appear in only one document.

Approach

Describe which indexing algorithm you chose. Describe which data structures you used. Which classes did you use for your data structures?

We used an in-memory indexing approach with Flex, C++, and hash tables. Each document is tokenized, and a document hash table counts the frequency of each term. After processing a document, the program updates a global hash table and stores the document ID and term frequency in a C++ map of posting lists.

Once all documents have been processed, the program writes the dictionary and posting files, excluding terms that appear in only one document. The mapping file stores the filenames associated with each document ID.

The main classes used were HashTable, dictFile, postFile, mapFile, and invertedFile. The hash tables use linear probing to handle collisions.

Results

Describe the record contents and sizes of the fields in your dict and post and map files for your largest document collection.

Dict file:

Term (bytes)	**Numdocs (bytes)**	**Start (bytes)**	**Whitespace (bytes)**	**TotalRecord(bytes)**
14	**4**	**6**	**3?**	**27**

Post file:

Docid (bytes)	**Freq (bytes)**	**Whitespace (bytes)**	**TotalRecord(bytes)**
4	**8**	**2?**	**14**

Map file:

Filename (bytes)	**Whitespace (bytes)**	**TotalRecord(bytes)**
30	**1?**	**31**

unique tokens in your largest document (with filtering long tokens):
___42,139_____

Size of your document hashtable: __300,009_____

unique tokens in your final collection (with filtering long tokens): __160,997_____

Size of your global hashtable ___300,009_____

Show the complete dict, post and map files for the tinyfiles collection (**use a small globalht size**, e.g., 21 buckets/7 keys)

Show the first 50 records of dict and 10 post and map files for the large files collection.

Confirm that your files are fixed length:
Results of tinyfiles Testcase

	Dict	**Post**	**Map**
NumRecords from wc -l	**3**	**10**	**4**
Filesize from ls -l	**81**	**140**	**124**
Observed RecordSize: Filesize/NumRecords	**27**	**14**	**31**

Does the observed RecordSize match your predicted TotalRecord size? Yes or No

Results of Full files Testcase

	Dict	**Post**	**Map**
NumRecords from wc -l	**57,391**	**859,021**	**1,200**
Filesize from ls -l	**1,549,557**	**12,026,294**	**37,200**
Observed RecordSize: Filesize/NumRecords	**27**	**14**	**31**

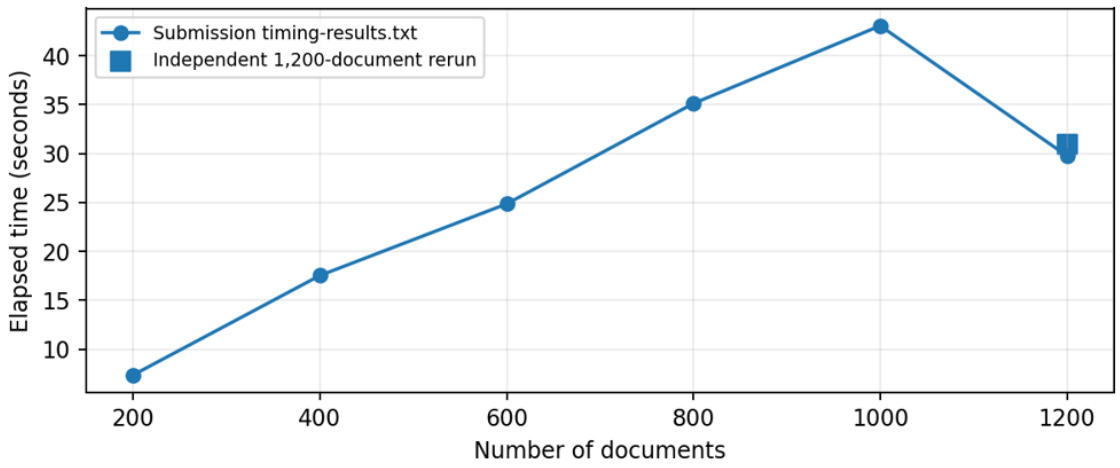
Does the observed RecordSize match your predicted TotalRecord size? Yes or No

Efficiency
Timings

Homework 1 processed the same 1,200-document collection in 16.767 seconds. An independent full run of the updated HW3 Part III code took 30.896 seconds, or 14.129 seconds longer (approximately 1.84 times the HW1 time). HW3 does additional work to maintain hash tables, posting lists, and three index files, so its elapsed time is not a like-for-like tokenizer-only benchmark.

The six batch measurements below are taken directly from the updated submission's timing-results.txt. The 1,200-document time is lower than the 1,000-document time, likely reflecting run-to-run conditions; it should not be interpreted as proof that processing more documents is generally faster.

Documents	Real time (seconds)
200	7.368
400	17,531
600	24,853
800	35,115
1000	43,051
1200	29,765



Complexity

I/O: Let C be the number of input characters. Reading/tokenizing the documents takes O(C). Writing the dictionary, postings, and mapping files adds O(U + P + D) fixed-size records, where U is retained vocabulary size, P is the number of retained document–term pairs, and D is the number of documents. Overall I/O is O(C + U + P + D), usually summarized as O(C) when output size is bounded by input size.

In-memory processing: Let N be the number of accepted tokens, U the number of distinct terms, P the number of document–term pairs, D the number of documents, and H the per-document hash-table bucket count. Expected hash insert/lookup work is O(N). Ordered std::map posting-list insertion costs O(log U) per distinct document–term pair, and scanning the full document hash table contributes O(DH). Thus an implementation-aware expected bound is O(N + P log U + DH), plus O(U + P) to emit index records. With a fixed H, the dominant term is approximately O(N + P log U).

Testing

For the word **razorback**:
What record number is *razorback* in the dict file (remember to count from 0)

What is the **dict** entry for *razorback*:
Token: __razorback__
Start: __599076__
NumDocs: __7__

What is the **post** entry for the record whose number is filled in for Start above (remember to count from 0):
Docid: __70__
Freq: __1__

What is the **map** entry for the record whose number is filled in for Docid above (remember to count from 0):
Filename: ___uark_30.html___

How many times does *razorback* appear in the that document (use **grep**)?
1 time

Check	Observed result
Dictionary record number (zero-based)	191,588
Dict token	razorback
Dict start posting (zero-based)	599,076
Dict NumDocs	7
Post at record 599,076 — DocID	70
Post at record 599,076 — Frequency	1
Map at record 70	Filename: uark_30.html
Actual source filename	../files_updated/files/uark_30.html
grep -oiw razorback … | wc -l	1

The source-document grep count of 1 agrees with the posting frequency of 1.

Appendix (Last 30 records from full collection)

Last 30 dict records:

0 0
0 0
0 0
0 0
0 0
0 0
0 0

0 0
0 0
0 0
0 0
0 0
0 0
0 0
0 0
pulmonary 6 854520
0 0
castled 2 854526
0 0
0 0
execute 12 854528
bantering 2 854540
0 0
0 0
castle 27 854542
0 0
0 0
0 0
0 0
0 0

Last 30 post records:

1146 1
85 1
979 1
23 49
85 2
170 2
284 3
286 7
328 2
454 1
464 1
555 4
666 8
715 1
721 1
741 8
770 1

787 1
820 2
822 3
866 9
875 1
887 2
979 2
986 7
987 7
1003 17
1065 1
1104 2
1120 6

Last 30 map records:

vanderbilt_1.html
cmu_10.html
jhu_12.html
uci_17.html
wisc_21.html
jhu_16.html
brown_30.html
msu_6.html
wustl_1.html
osu_5.html
umn_16.html
cornell_12.html
umich_6.html
gatech_15.html
utexas_2.html
caltech_20.html
msu_17.html
caltech_1.html
msu_19.html
gatech_19.html
northwestern_28.html
stanford_5.html
washington_2.html
upenn_13.html
colorado_29.html
utexas_10.html
washington_27.html

illinois_5.html
brown_15.html
osu_2.html